

Relazione tecnica

Indice

1. Introduzione
2. API NASA utilizzate e motivazione scelta
3. Architettura dell'applicazione
 - a. Tecnologie software scelte
 - i. Sqlite3
 - ii. Python3
 - iii. Flask
 - iv. Html, css, javascript
 - v. docker
 - b. Struttura del database
 - i. Entita' necessarie
 - ii. Campi delle entita'
 - iii. Normalizzazione dei dati
 - iv. Tabelle risultanti
 - c. Funzionamento del codice backend
 - i. Rotte verso pagine
 - ii. Rotte verso servizi (API locali)
 - iii. Logica lato rete (client)
 - iv. Logica lato base dati (db)
 - v. Logica di orchestrazione (servizi)
 1. Importare il progetti NASA
 2. Aggiornare i progetti NASA
 3. Eliminare i progetti NASA
 4. Inviare i progetti in formato json
 5. Inviare il progetto richiesto in formato json
 - d. Funzionamento del codice lato utente
 - i. Pagine viste dall'utente
 - ii. Dati visualizzabili
 - iii. Interazioni sui dati
4. Principali queries utilizzate
5. Osservazioni sull'analisi dei dati
6. Soddisfacimento dei requisiti tecnici
 - a. Uso di api NASA
 - b. Uso di un database sql
 - c. Query sql manuale create
 - d. Query sql manuale read (select)
 - e. Query sql manuale update

- f. Query sql manuale delete
- g. Query sql manuale analitica (join o funzioni aggregazione)
- h. Dashboard con correlazioni tra proprieta' progetti
- i. Possibilita' di filtro dei progetti
 - i. Filtro per Periodo
 - ii. Filtro per Categoria
 - iii. Filtro per Stati coinvolti
 - iv. Filtro per Destinazione
- 7. Implementazione di bonus facoltativi
 - a. Mappa US interattiva per applicazione filtri

Pseudocodice dei 5 servizi backend

```

services.import_projects()
  db.create_database_if_not_exists()
  attributes = client.fetch_pjs_attributes()
  projects = client.fetch_relevant_projects()
  db.populate_database(attributes, projects)
  count the rows
  return the success json

services.update_projects()
  update pjs attributes
    client.fetch_pjs_attributes()
    delete all rows in attributes table
    db.populate_attributes_tables()
  fetch relevant pjs
  fetch nonce
  fetch projects/search using nonce
    return only necessary pjs fields
    sort by viewCount
    take first 3k
  update projects table
    mem added
    mem updated
    mem removed
    for each updated relevant pj
      if existing in pjs table
        update each field

```

```

                updated +1
            else
                insert pj
                added +1
            sort pjs table desc
            remove n=added pjs from bottom to up
        update junctions tables
        delete all rows in junctions tables
        populate junctions tables
        log the db update(added, updated, deleted)
        return last database update row in json

```

```

services.delete_projects()
    delete all db tables rows
    redirect to import.html page

```

```

services.get_projects(filters=null)
    read filters from url params
    sql select join query
    parse json projects
    return projects.json

```

```

services.get_project(id)
    read the id from url path
    sql select join for the id
    parse project in json
    return project.json

```

Albero delle cartelle dell'applicazione

```

nasa-dashboard/
  .venv
  app/
    .env
    .env.example
    requirements.txt
    main.py
      GET /
      GET /import
      GET /projects?{projects_state}

```

```

    GET /projects/{id}
    POST /api/projects
    PUT /api/projects
    GET /api/projects?{projects_state}
    DELETE /api/projects
    GET /api/projects/{id}?{projects_state}
services.py
    import_projects()
    update_projects()
    delete_projects()
    get_projects(filters=null)
    get_project(id)
client.py
    fetch_nasa_pjs_attributes()
        fetch_status_values()
            GET /api/enums "projectStatusType":[]
        fetch_technologies()
            GET /api/taxonomies/8817
taxonomies-8817-response
    fetch_destinations()
        GET /api/enums "destinationTypes":[]
    fetch_nasa_relevant_projects()
        fetch_nonce()
        POST with payload for filter
            return only necessary pjs fields
            sort by viewCount
            take first 3k
db/
    Schema.sql
    db.py
        create_db()
        populate_db()
            populate_attributes_tables()
                populate_status(statuses)
                populate_technologies(technologies)
                populate_states
                    hard coded loop for us states
                populate_destinations(destinations)
            populate_projects_table()
            populate_junctions_tables()
            update_pjs_attributes_tables()

```

```
        update_projects_table()
        update_junction_tables()
static/
    styles/
        js/
            import.js
            projects.js
            project.js
    templates/
        import.html
        projects.html
        project.html
        log.html
data/
    database.db
.gitignore
Dockerfile
compose.yaml
README.md
```